

Architecture of an Agentic QE Pipeline

The Evolution of Test Automation Traditional Quality Engineering relies heavily on manual translation: a product owner writes a user story, a QA engineer reads it, brainstorms scenarios, and manually types out test cases. An Agentic AI pipeline fundamentally disrupts this by treating the entire lifecycle as an automated data pipeline. Rather than just generating code, autonomous AI agents act as a bridge between your project management software and your test management repository.

The Core Components A fully realized Agentic AI framework for QE consists of three distinct phases:

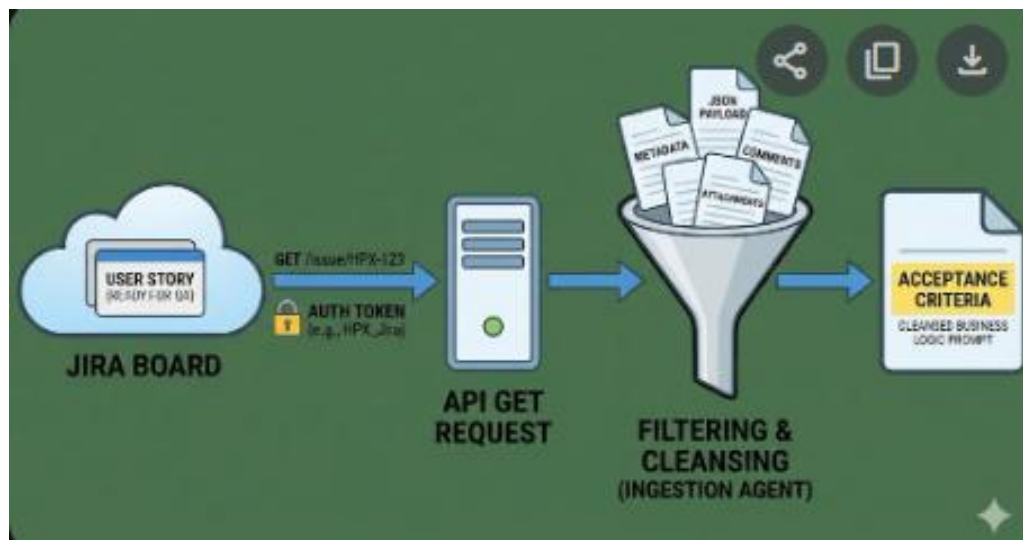
1. **Ingestion & Parsing:** Extracting the raw, unstructured text from a user story.
2. **Cognitive Generation:** Using Large Language Models (LLMs) to logically deduce scenarios and step-by-step cases.
3. **Structuring & Publishing:** Formatting the AI's output into a strict schema and pushing it to a central repository.

This approach ensures absolute traceability, mapping every generated test step directly back to the original business requirement.

Step 1 - Unstructured Data Ingestion

Connecting to the Source of Truth The pipeline begins the moment a user story is flagged as "Ready for QA". The first agent in the framework, the "Ingestion Agent," is responsible for gathering context. It cannot rely on the user story title alone; it must pull the description, acceptance criteria, attachments, and even historical comments to fully understand the feature's intent.

API-Driven Extraction To achieve this, the Ingestion Agent connects directly to the issue tracking system's API. The integration module securely authenticates using your configured authorization token (e.g., HPX_Jira) to fetch the JSON payload of the target user story. The agent then cleanses this payload, stripping out irrelevant metadata and isolating the core business logic, creating a clean, high-density prompt for the next phase of the pipeline.



Step 2 - The Scenario Planner Agent

From Feature to Coverage Matrix Once the cleaned user story is ingested, it is passed to the "Planner Agent." This AI model is specifically prompted with a persona (e.g., "Senior Quality Architect") and is tasked with generating a comprehensive test coverage matrix. It does not write test steps yet; it focuses purely on *what* needs to be tested.

Categorizing Scenarios The Planner Agent breaks the requirement down into specific testing categories:

- **Happy Path (Positive):** Ensuring the core functionality works exactly as designed.
- **Error Handling (Negative):** Intentionally providing invalid inputs to verify graceful failures.
- **Boundary Values:** Testing the absolute limits of numerical or character constraints.
- **Security/Access:** Verifying that only authorized user roles can trigger the feature.

This scenario matrix is outputted as a structured list, serving as the blueprint for the actual test cases.



Step 3 - Structuring & Publishing Test Cases

Formulating the Steps With the test scenarios defined, the "Authoring Agent" takes over. This agent translates the conceptual scenarios into actionable, step-by-step instructions. For every scenario, it generates:

- **Pre-conditions:** The system state required before testing.
- **Test Steps:** Clear, imperative actions (e.g., "Click the submit button").
- **Expected Results:** The exact, verifiable outcome of the action.

Automated Publishing Because test management tools require strict formatting, the Authoring Agent formats its output into a standardized JSON schema. The "Publisher Agent" then takes this payload, authenticates against the test management system using the appropriate API key (e.g., HPX_TestRail), and utilizes POST requests to automatically create the test cases within the designated project suite. This eliminates manual data entry entirely.

